Comprehensive Project Briefing & Theory Guide

Bridging the Sim-to-Real Gap: Reinforcement Learning for Recommender Systems via *In Silico* LLM Personas

Brandon Tran (M.S. S&DS)
brandon.tran@yale.edu

October 20, 2026

Document Purpose: This briefing provides an end-to-end, self-contained technical and conceptual guide to the project. It bridges foundational intuitions and analogies with formal mathematical theory, contrasting existing research against our proposed framework, detailing our computational acceleration, and specifying how success is evaluated.

---

1. Conceptual Foundations: Recommendation as Sequential Control

The Cafeteria Analogy (Junk Food vs. A Balanced Diet) To understand why sequential decision making matters in recommendation, consider a dining hall manager catering to a student:

- **The Myopic Ranker (Today's RecSys):** Observes that the student loves French fries. The algorithm serves French fries for breakfast, lunch, and dinner because the instantaneous probability of the student eating them right now is 99%. By day four, the student suffers acute nutritional fatigue, feels sick of potatoes, and stops visiting the cafeteria entirely.
- **The Sequential Controller (Reinforcement Learning):** Recognizes that while fries yield high immediate pleasure, long-term health and customer retention require balanced variety. It deliberately serves grilled salmon or a salad today—accepting a slight drop in immediate delight—to ensure the student remains healthy, satisfied, and returning for months.

What Commercial Recommenders Do Today In industrial pipelines (e.g., Netflix, YouTube, TikTok), recommenders decompose the problem into point-wise candidate scoring using deep collaborative filtering models (e.g., Two-Tower networks, DLRM). They score items independently:

$$\hat{y}_{u,i} = f(\boldsymbol{x}_u, \boldsymbol{x}_i) \approx P(\text{Click} \mid \text{User } u, \text{Item } i) \tag{1}$$

The policy greedily selects the top-K candidates that maximize instantaneous expected reward:

$$\pi_{\text{myopic}} = \arg\max_{i \in \mathcal{A}} \mathbb{E}[R_t \mid a_t = i] \tag{2}$$

The Fatal Flaw: Point-wise scoring assumes state transitions are static ($\mathcal{P}(s_{t+1} \mid s_t, a_t) = s_t$). If a user watches a thriller, greedy rankers flood their feed with identical thrillers. Over multiple recommendations, the user's marginal utility for that genre collapses:

$$\frac{\partial U}{\partial n_{\text{genre}}} \longrightarrow 0 \tag{3}$$

This induces acute **semantic satiation** (genre fatigue), trapping users in repetitive filter bubbles and accelerating session churn.

The Reinforcement Learning Formulation Reinforcement Learning models recommendation as an agent interacting with a dynamic user environment over time. Instead of maximizing the instantaneous reward R_t, the RL agent maximizes the **cumulative discounted return**:

$$G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k} \tag{4}$$

where $\gamma \in [0, 1)$ is the discount factor. When $\gamma \to 1$, the agent is incentivized to balance immediate engagement with long-term retention—sacrificing a sub-optimal click today to prevent user burnout next week.

---

2. The Core Challenge: The RL Sim-to-Real Dilemma

If reinforcement learning is mathematically superior, why isn't every recommendation platform powered by it? In robotics or games (e.g., AlphaGo), an RL agent can fail millions of times in simulation without cost. In commercial RecSys, three obstacles prevent deployment:

1. **Live A/B Testing is Prohibitive:** RL algorithms must *explore* sub-optimal actions to discover long-horizon strategies. Deploying exploratory policies on real human subscribers causes bad recommendations, leading to direct app uninstalls, brand damage, and permanent subscriber churn.

2. **Offline Logs Suffer from Counterfactual Exposure Bias:** Static historical logs $\mathcal{D} = \{(s_t, a_t, r_t)\}$ only record feedback for items the historical logging policy chose to expose ($O_{u,i} = 1$). We have zero observation of how a user would react to counterfactual, unshown items. Offline algorithms like **Conservative Q-Learning (CQL)** add a penalty against unseen actions:

$$\min_Q \alpha \left(\mathbb{E}_{s\sim\mathcal{D}, a\sim\mu}[Q(s,a)] - \mathbb{E}_{(s,a)\sim\mathcal{D}}[Q(s,a)] \right) + \frac{1}{2}\mathbb{E}\left[\left(Q - \mathcal{B}^\pi \hat{Q} \right)^2 \right] \tag{5}$$

   In catalogs with $|\mathcal{A}| > 100,000$ items, over 99.99% of actions are unobserved. CQL becomes excessively timid and collapses into merely cloning the historical logging heuristic.

3. **Toy Simulators Suffer from a Severe Sim-to-Real Gap:** To enable safe interaction, researchers built simulators like Google's **RecSim**. However, RecSim hand-codes user transitions using rigid linear decay formulas:

$$\theta_{t+1} = \gamma\theta_t + \eta \boldsymbol{x}_{a_t} \tag{6}$$

   Deep RL agents are optimization machines: they quickly discover mathematical artifacts in these hand-crafted equations, learning exploitative policies that score high in the simulator but fail completely when transferred to real humans.

---

3. Existing Research vs. Our Proposed Framework

To clarify the contribution, we contrast existing research paradigms against our proposed setup:

Dimension	Parametric Simulators (RecSim)	Generative Agents (Agent4Rec)	Our Proposed Framework ($\mathcal{M}_{\text{LLM}}$)
User Dynamics	Hand-crafted linear decay formulas	High-capacity LLM cultural priors	**High-capacity LLM cultural priors**
Role of LLM	None	Static post-hoc evaluation judge	**Closed-loop interactive Gym environment**
Action Space	Discrete / Toy continuous	Fixed discrete candidate sets	**Continuous latent vector $\boldsymbol{a}_t \in \mathbb{R}^d$ + MIPS**
Step Latency	< 1 ms	$\sim$ 450 ms (sequential decoding)	**< 12 ms (Zero-generation logit scoring)**
RL Feasibility	Fast, but fails on real users	Computationally impossible	**Practical (10^5 steps in 20 min on HPC)**

The Missing Link: Recent work (Agent4Rec, Wang et al., 2024; Generative Agents, Park et al., 2023) showed that LLMs possess deep commonsense knowledge of human tastes. However, in all existing literature, **LLMs have only been used as static evaluators** to benchmark other algorithms after the fact. Nobody has closed the loop to make the LLM the interactive transition operator inside an RL training loop due to prohibitive inference latency. We resolve this bottleneck.

---

4. Formal Mathematical Framework (POMDP Formulation)

Real users never expose their internal thoughts. Recommendation is fundamentally a **Partially Observable Markov Decision Process (POMDP)**, formalized as the 7-tuple:

$$\mathcal{M} = \langle \mathcal{S}, \mathcal{A}, \mathcal{T}, \mathcal{R}, \Omega, \mathcal{O}, \gamma \rangle \tag{7}$$

- **Hidden State $s_t \in \mathcal{S}$:** The user's true cognitive state at step t:

$$s_t = \left(\boldsymbol{u}_{\text{taste}}, \text{fatigue}_t, \text{mood}_t, \text{patience}_t\right) \tag{8}$$

  This state is completely unobservable to the recommendation agent.

- **Observed History $o_t \in \Omega$:** The observable trajectory emitted by observation function $\mathcal{O}(s_t)$:

$$o_t = \left(\boldsymbol{x}_{\text{profile}}, h_{1:t-1}\right) = \left(\boldsymbol{x}_{\text{profile}}, (i_1, r_1), (i_2, r_2), \ldots, (i_{t-1}, r_{t-1})\right) \tag{9}$$

  The policy $\pi(a_t \mid o_t)$ must implicitly maintain a belief state over user satisfaction.

- **Continuous Action Space $\boldsymbol{a}_t \in \mathbb{R}^d$:** In a catalog of 20,000 movies, discrete action spaces create an intractable branching factor where computing $\max_{a \in \mathcal{A}} Q(s, a)$ requires 20,000 forward passes per step. Instead, our policy outputs a continuous target vector $\boldsymbol{a}_t \in \mathbb{R}^{64}$. The actual catalog item i_t served to the user is determined via Maximum Inner-Product Search (MIPS):

$$i_t = \arg \max_{i \in \text{Catalog}} \frac{\langle \boldsymbol{a}_t, \boldsymbol{e}_i \rangle}{\|\boldsymbol{a}_t\| \|\boldsymbol{e}_i\|} \tag{10}$$

  This allows us to train using **Soft Actor-Critic (SAC)**, a sample-efficient actor-critic algorithm that optimizes the maximum entropy objective:

$$J(\pi) = \sum_{t=0}^{T} \mathbb{E}_{(s_t, a_t) \sim \rho_\pi} \left[R(s_t, a_t) + \alpha \mathcal{H}(\pi(\cdot \mid s_t)) \right] \tag{11}$$

- **Coupled Step Reward R_t:** To prevent the policy from burning out the user to capture an immediate high rating, our reward balances consumption quality with session survival:

$$R_t = \underbrace{\frac{r_t - 1}{4}}_{\text{Normalized Rating} \in [0,1]} + \underbrace{0.25 \cdot \mathbb{I}(c_t = 1)}_{\text{Retention Bonus}} \tag{12}$$

  where $r_t \in \{1, \ldots, 5\}$ is the simulated rating and $c_t \in \{0, 1\}$ indicates whether the user continues browsing.

---

5. Behavioral Signatures & Pre-RL Persona Validation

An LLM simulator is only as reliable as its behavioral grounding. Rather than prompting with qualitative text (e.g., "You are an action movie fan"), we extract quantitative empirical priors from historical MovieLens splits:

1. **Empirical Genre Entropy:** Quantifies user specialization versus breadth:

$$H(G) = -\sum_{g \in \text{Genres}} p_g \log p_g \tag{13}$$

2. **Historical Rating Variance:** Captures whether the user is generous or critical:

$$\sigma_r^2 = \frac{1}{|H_u|} \sum_{r \in H_u} (r - \bar{r}_u)^2 \tag{14}$$

3. **Popularity Percentile:** The ratio of mainstream hits versus niche indie cinema.

Satiation via Attention In traditional simulators, satiation is modeled via rigid equations. In our framework, satiation is captured organically through the transformer's **self-attention mechanism**:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V \tag{15}$$

As identical genres accumulate in the prompt context window, the model's cross-token attention heads automatically register repetition and adjust expected rating logits downward without hard-coded rules.

Pre-RL Validation Suite Before training RL policies, we validate persona fidelity against held-out user logs:

- **Rating Calibration (ECE & MSE):** Ensure simulated rating distributions match empirical test ratings.
- **Distributional Alignment:** Measure Wasserstein distance $\mathcal{W}_1(P_{\text{sim}}, P_{\text{real}})$ across user trajectories.
- **Temporal Stability:** Run identical multi-step rollouts to verify that personas do not experience stochastic drift.

---

6. Computational Acceleration: Logit Scoring & KV Caching

The fundamental barrier to using LLMs as RL environments is inference latency. Standard autoregressive generation requires ~ 450 ms per step, requiring over 120 hours of GPU time for 10^5 interaction steps. We overcome this through two complementary techniques:

1. Zero-Generation Token-Logit Scoring We format the prompt so that the user's rating is deterministically the very next single token:

```
Recommended Title:  Inception (Sci-Fi, Heist) ...Your rating (1-5) is:
```

Setting `max_tokens` $= 1$ eliminates autoregressive decoding completely. In a single forward prefill pass, we extract the unnormalized logits corresponding to tokens "1" through "5":

$$P_{\text{LLM}}(r_t = k \mid \text{context}) = \frac{\exp(\ell_k/\tau)}{\sum_{j=1}^{5} \exp(\ell_j/\tau)}, \quad k \in \{1, \ldots, 5\} \tag{16}$$

This yields a closed-form expected rating $\mathbb{E}[r_t] = \sum_{k=1}^{5} k \cdot P_{\text{LLM}}(r_t = k)$ that is exact, differentiable, and has zero string-parsing failure.

2. Archetype Clustering & Prefix Caching In online RL, we run 16 to 32 parallel worker environments. We cluster users into representative behavioral archetypes:

- Dozens of parallel workers share identical system instructions and persona schema prefixes.
- Using **vLLM PagedAttention**, the Key-Value (KV) cache tensors for the persona prefix are pinned permanently in GPU memory.
- **Prefix Cache Hit Rate = 100%:** Each environment step only processes the small suffix containing the recommended movie title.
- **Throughput:** Drops step latency from $\sim$ **450** ms $\rightarrow <$ **12** ms on an NVIDIA A100 GPU on Yale's Bouchet HPC cluster—a $> 35\times$ speedup that enables $100,000$ RL steps in under 20 minutes.

---

7. Evaluation Protocol, Comparison Baselines & Defining Success

The Three-Way Comparison Benchmark To evaluate whether generative world models mitigate the sim-to-real gap, we train three policies under identical continuous action representations ($\boldsymbol{a}_t \in \mathbb{R}^d$):

1. $\pi_{\textbf{RecSim}}$ **(Parametric Baseline):** Continuous SAC trained inside Google's RecSim with handcrafted decay rules.
2. π_{CQL} **(Offline Baseline):** Conservative Q-Learning trained directly on static historical logs.
3. $\pi_{\textbf{LLM}}$ **(Proposed Generative Setup):** Continuous SAC trained inside our interactive LLM persona environment $\mathcal{M}_{\text{LLM}}$.

The Sim-to-Real Transfer Test We freeze all three policies and evaluate their cumulative discounted return zero-shot on held-out real user trajectories $\mathcal{D}_{\text{real}}$. Theoretical bounds dictate that simulation transfer error is bounded by the Total Variation divergence between transition dynamics:

$$\left| V^{\pi}_{\mathcal{M}_{\text{sim}}} - V^{\pi}_{\mathcal{M}_{\text{real}}} \right| \leq \frac{2\gamma R_{\max}}{(1-\gamma)^2} \cdot D_{\text{TV}} \left(\mathcal{P}_{\text{sim}}(\cdot \mid s,a) \;\|\; \mathcal{P}_{\text{real}}(\cdot \mid s,a) \right) \tag{17}$$

Our central scientific hypothesis is that the high-capacity LLM world model achieves a dramatically tighter divergence bound than rigid linear decay models:

$$D_{\text{TV}}(\mathcal{P}_{\text{LLM}} \,\|\, \mathcal{P}_{\text{real}}) \ll D_{\text{TV}}(\mathcal{P}_{\text{RecSim}} \,\|\, \mathcal{P}_{\text{real}}) \tag{18}$$

The KuaiRec Dense Matrix Safeguard Collaborative filtering datasets (MovieLens-1M) have high sparsity ($> 95\%$ unobserved ratings), which can introduce variance into off-policy evaluation. If MovieLens evaluations prove noisy, we evaluate on **KuaiRec**, which provides a fully observed $1,411 \times 3,327$ interaction matrix where every user evaluated every video, serving as an uncorrupted counterfactual oracle.

What Success Looks Like

- **Fidelity Milestone:** Personas achieve $R^2 \geq 0.80$ and Wasserstein distance $\mathcal{W}_1 \leq 0.35$ on held-out MovieLens ratings.
- **Computational Milestone:** Sustained throughput > 65 environment steps/second (< 15 ms/step) on Yale's Bouchet HPC.
- **Policy Superiority Milestone:** The generative policy significantly outperforms both the parametric and offline baselines on real test logs:

$$V^{\pi_{\text{LLM}}}_{\text{real}} > V^{\pi_{\text{RecSim}}}_{\text{real}} \quad \text{and} \quad V^{\pi_{\text{LLM}}}_{\text{real}} > V^{\pi_{\text{CQL}}}_{\text{real}} \quad (p < 0.01) \tag{19}$$

- **Exploration Milestone:** The policy demonstrates higher genre entropy than CQL, proving it learns proactive, non-myopic exploration without user fatigue.

---

8. Anticipated Q&A & Conceptual Defense

Q1: Isn't an LLM just hallucinating human taste? *Response:* Unconstrained LLMs hallucinate when asked to generate arbitrary text. In our architecture, the LLM is tightly conditioned on quantitative historical priors (genre entropy, rating variance, and historical view percentiles), and output is mathematically restricted to logit extraction over tokens 1 through 5. Furthermore, our methodology enforces a Pre-RL Validation phase where we measure calibration error against held-out ground truth before running any RL training.

Q2: Why use continuous actions instead of discrete top-K ranking? *Response:* In a catalog of tens of thousands of items, discrete action spaces ($\binom{|\mathcal{A}|}{K}$) make Q-learning and policy gradients intractable. Outputting a continuous embedding vector $\boldsymbol{a}_t \in \mathbb{R}^d$ and using Maximum Inner Product Search (MIPS) to retrieve items allows us to use Soft Actor-Critic (SAC)—which is provably more stable, sample-efficient, and capable of exploring smooth latent taste spaces.

Q3: How does this scale to real production systems? *Response:* This framework is designed as an **offline training sandbox**, not a real-time live inference engine. Just as aerospace engineers use wind tunnels and flight simulators to train autonomous autopilots before flying a commercial airplane, RecSys engineers can use our generative *in silico* environment to train and stress-test continuous RL agents on GPU clusters before deploying them to live production traffic.